Spam Email Classifier

Step-by-step prompt guide using Naive Bayes and Support Vector Machines (SVM)

Project Overview

The goal is to train a machine learning model that reads the content of an email or message and decides whether it is **Spam** or **Ham** (legitimate). This is a binary text classification problem and a classic Natural Language Processing (NLP) project. The guide splits the work into five phases, each with an objective and a ready-to-use prompt for an AI coding assistant.

Phase	What you do	Main tools
1. Data & EDA	Load dataset, check balance and message lengths	Pandas, Seaborn
2. Preprocessing	Clean text, split data, build TF-IDF features	NLTK, Scikit-Learn
3. Training	Train Multinomial Naive Bayes and SVM	Scikit-Learn
4. Evaluation	Precision, recall, F1, confusion matrix	Scikit-Learn, Matplotlib
5. Inference	Save model, build classify_email(text)	Joblib

Setup (before Step 1)

```
pip install pandas numpy scikit-learn nltk seaborn matplotlib joblib
```

Recommended datasets

- **SMS Spam Collection (UCI):** about 5,500 labeled messages, ideal for a first run.
- **Enron Spam Dataset:** real full-length emails, closer to production use.
- **SpamAssassin Public Corpus:** mixed spam and ham email with headers.

Why these two models?

- **Multinomial Naive Bayes:** very fast, works well on small data, a strong baseline for text.
- **SVM (linear kernel):** finds the best separating hyperplane and often gives higher accuracy on TF-IDF features.

Tip: run the prompts in order and paste the previous step's output (variable names, errors) into the next prompt so the assistant keeps your code consistent.

STEP 1 Dataset Collection and Exploration

Objective: Load a standard spam dataset and run Exploratory Data Analysis (EDA) to understand class balance and text length.

PROMPT TO USE

"Write Python code using Pandas and Seaborn to load the SMS Spam Collection dataset (or generate a placeholder structure if no file path is provided). Show me how to check for missing values, analyze the class distribution (spam vs. ham percentages), and visualize the distribution of email/message lengths for both classes."

What to look for

- **Missing values and duplicates:** drop or fix them before modeling.
- **Class imbalance:** spam is usually only 10-15% of data, so accuracy alone will mislead you.
- **Length differences:** spam tends to be longer and contains more digits and capital letters.
- **Label encoding:** map ham to 0 and spam to 1 for scikit-learn.

Starter code skeleton

```
import pandas as pd, seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv('spam.csv', encoding='latin-1')[['v1','v2']]
df.columns = ['label', 'text']
print(df.isnull().sum())
print(df['label'].value_counts(normalize=True) * 100)

df['length'] = df['text'].str.len()
sns.histplot(data=df, x='length', hue='label', bins=50)
plt.show()
```

Follow-up prompts (optional)

- "Add a word cloud for spam and ham messages separately."
- "Show the top 20 most frequent words in each class as bar charts."
- "Add features for digit count, capital-letter ratio and presence of URLs, and compare them across classes."

STEP 2 Text Preprocessing and Feature Extraction

Objective: Clean raw text and convert words into numerical features using TF-IDF (or Bag of Words).

PROMPT TO USE

"Write a Python preprocessing function using NLTK or Scikit-Learn that takes raw email text, converts it to lowercase, removes HTML tags, punctuation, stop words, and applies lemmatization. Then, show me how to split the dataset into training and testing sets and transform the text data into numerical features using TfidfVectorizer."

Cleaning checklist

- Lowercase, strip HTML tags, remove punctuation and extra whitespace.
- Remove stop words (the, is, and) and lemmatize ('winning' and 'wins' become 'win').

Splitting and vectorizing

- Use `train_test_split(test_size=0.2, stratify=y, random_state=42)` so both sets keep the same spam ratio.
- **Fit the TfidfVectorizer on training data only**, then transform the test data. Fitting on the full dataset causes data leakage.
- Good starting settings: `max_features=5000`, `ngram_range=(1,2)`, `min_df=2`.

Starter code skeleton

```
import re, nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

lem = WordNetLemmatizer(); stop = set(stopwords.words('english'))

def preprocess(text):
    text = re.sub(r'<.*?>', ' ', text.lower())
    text = re.sub(r'[^a-z\s]', ' ', text)
    return ' '.join(lem.lemmatize(w) for w in text.split() if w not in stop)

df['clean'] = df['text'].apply(preprocess)
X_train, X_test, y_train, y_test = train_test_split(
    df['clean'], df['label'], test_size=0.2, stratify=df['label'], random_state=42)
vec = TfidfVectorizer(max_features=5000, ngram_range=(1,2))
X_train_tfidf = vec.fit_transform(X_train)
X_test_tfidf = vec.transform(X_test)
```

Note: run nltk.download('stopwords') and nltk.download('wordnet') once before using this code.

STEP 3 Model Training: Naive Bayes vs. SVM

Objective: Train a fast baseline (Naive Bayes) and a stronger model (SVM) on the same TF-IDF features.

PROMPT TO USE

"Write Python code using Scikit-Learn to train two models on my TF-IDF feature matrices: a Multinomial Naive Bayes classifier and a Support Vector Classifier (SVC). Include code to train both models on the training set and make predictions on the test set."

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC

nb = MultinomialNB(alpha=0.1).fit(X_train_tfidf, y_train)
svm = SVC(kernel='linear', C=1.0, probability=True).fit(X_train_tfidf, y_train)

nb_pred = nb.predict(X_test_tfidf)
svm_pred = svm.predict(X_test_tfidf)
```

Hyperparameters worth tuning

- **Naive Bayes:** alpha (smoothing), try 0.01 to 1.0.
- **SVM:** C (regularization) and kernel (linear usually best for text); LinearSVC is faster on large data.
- Use `GridSearchCV(cv=5, scoring='f1')` to pick values automatically.

STEP 4 Model Evaluation and Performance Metrics

Objective: Go beyond accuracy. Precision protects real emails from being filtered; recall catches spam.

PROMPT TO USE

"Write Python code to evaluate the performance of my trained Naive Bayes and SVM models. Generate a classification report (showing Precision, Recall, F1-score), a confusion matrix visualization using Matplotlib/Seaborn, and discuss which model is better suited for spam detection and why."

Step 4 continued: Reading the Metrics

Metric	Meaning in spam detection	Priority
Precision	Of emails flagged as spam, how many really were spam. Low precision means real emails get lost.	Very high
Recall	Of all real spam, how much was caught. Low recall means spam reaches the inbox.	High
F1-score	Balance of precision and recall; best single number on imbalanced data.	High
Accuracy	Overall correct predictions; misleading when spam is the minority.	Low

```
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns, matplotlib.pyplot as plt

for name, pred in [('Naive Bayes', nb_pred), ('SVM', svm_pred)]:
    print(name); print(classification_report(y_test, pred, target_names=['Ham','Spam']))
    sns.heatmap(confusion_matrix(y_test, pred), annot=True, fmt='d', cmap='Blues')
    plt.title(name); plt.show()
```

Which model wins?

Typically SVM edges out Naive Bayes in overall F1, while Naive Bayes trains in milliseconds and is a great lightweight option. Choose by your goal: if false positives (real mail marked as spam) are costly, pick the model with the higher precision.

STEP 5 Building an Inference / Prediction Pipeline

Objective: Save the best model and vectorizer, then expose a simple function for new emails.

PROMPT TO USE

"Write Python code to save the best-performing model and the fitted TfidfVectorizer to disk using joblib. Then, create a Python function classify_email(text) that loads the saved artifacts, preprocesses a new text string, and returns a prediction ('Spam' or 'Ham') along with the prediction probability."

```
import joblib
joblib.dump(svm, 'spam_model.pkl'); joblib.dump(vec, 'tfidf.pkl')

def classify_email(text):
    model = joblib.load('spam_model.pkl'); v = joblib.load('tfidf.pkl')
    X = v.transform([preprocess(text)])
    p = model.predict_proba(X)[0]
    return ('Spam' if p[1] > 0.5 else 'Ham'), round(max(p), 3)
```

Next steps: build a Flask/Streamlit web app, test on the Enron dataset, add cross-validation, and try Logistic Regression or a Transformer model for comparison.